



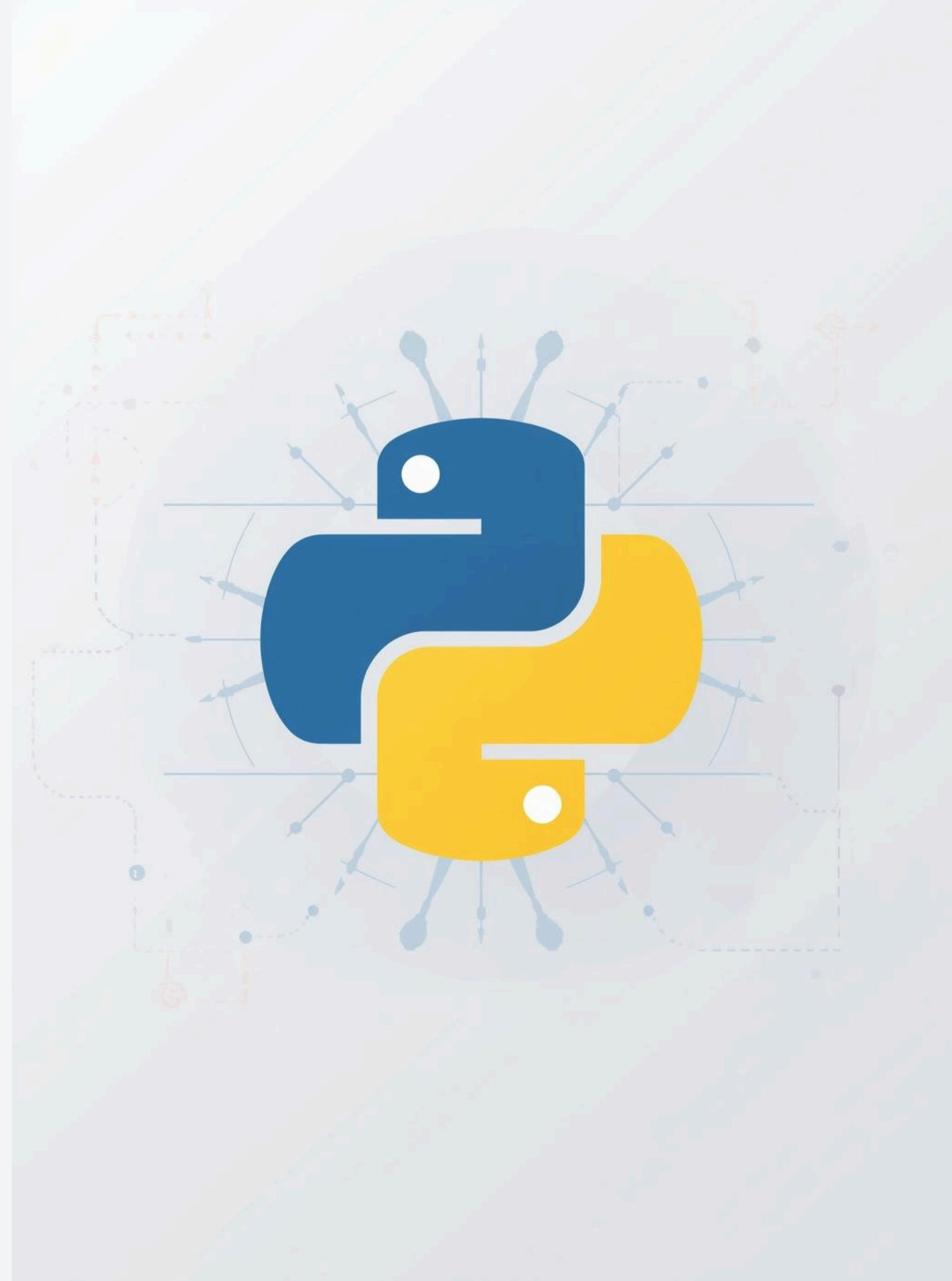
# Tkinter

Princípios e lógica de funcionamento

# O que é o Tkinter?

O Tkinter é a **biblioteca padrão** do Python para a criação de **Interfaces Gráficas de Usuário (GUI)**.

- Nativo e Leve: Já vem instalado com o Python, dispensando downloads extras.
- Onipresente: É a ferramenta mais utilizada para aprendizado e ferramentas internas.
- Cross-platform: O mesmo código funciona em Windows, macOS e Linux.
- Arquitetura "Wrapper": Ele atua como uma camada de interface (envoltório) sobre o Tcl/Tk.



# Por Dentro da Engrenagem

O Tkinter funciona como um tradutor entre duas camadas distintas da interface python e do sistema operacional da máquina

## TKINTER COMO TRADUTOR

O Tkinter atua como um **wrapper** entre o Python e o Tcl/Tk, facilitando a criação de interfaces gráficas ao traduzir comandos e eventos entre essas duas linguagens.

## TCL/TK E SO

O Tcl/Tk, um kit gráfico escrito em C, interage diretamente com o **sistema operacional**, permitindo que aplicações Python se comuniquem eficientemente com os componentes gráficos do ambiente.

# Vantagens e Desvantagens

## VANTAGENS

O Tkinter é **simples e leve**, ideal para iniciantes. Tornando sua curva de aprendizagem muito mais suave e rápida. Sua natureza multiplataforma permite que aplicações sejam executadas em diferentes sistemas operacionais sem modificações significativas no código. Possui estabilidade para uso em sistemas robustos e que exijam mais processamento.

## DESVANTAGENS

Apesar de suas vantagens, o Tkinter apresenta um **visual datado** e limitações em designs modernos (exigindo o uso de bibliotecas mais recentes para “maquiarem” o sistema desenvolvido), tornando-se menos adequado para aplicações que exigem interfaces sofisticadas e dinâmicas. Possui poucos recursos para estilização visual, tornando mais difícil criar designs “bonitos e modernos”. Por fazer processo de wrapper, não é indicado para sistemas que exijam um processamento visual muito pesado.

# A Lógica de Uso

A GUI funciona em ciclos de eventos, aguardando a interação do usuário a cada ciclo e adicionando ao ciclo seguinte.

O sistema é composto por containers que são inseridos ou sobrepostos a outro, a fim de construir a interface com o usuário.

## IMPORTAÇÃO DA BIBLIOTECA

Para começar a usar Tkinter, é necessário **importar a biblioteca** no seu código Python, permitindo o acesso a todas as suas funcionalidades.

```
import tkinter as tk
```

## CRIAÇÃO DA JANELA PRINCIPAL

A criação da janela principal é feita através da instância da classe Tk(), que representa a janela básica onde todos os widgets serão organizados e exibidos para o usuário.

```
root = tk.Tk()
```

## INSERÇÃO DE WIDGETS E LAYOUT

Dentro da variável "tk" é possível definir os elementos e posicioná-los, cumprindo com a hierarquia necessária.

```
label = tk.Label(root, text="Olá, Tkinter!")  
label.pack()
```

## EXECUÇÃO DO MAINLOOP

O método mainloop() é chamado para iniciar a aplicação, fazendo com que a janela permaneça aberta e aguardando interações do usuário, como cliques e entradas de texto.

```
root.mainloop()
```



# Gerenciadores de Layout

```
tk.Button(root, text="Relatórios").pack()  
tk.Button(root, text="Configurações").pack()
```

## PACK

O gerenciador Pack organiza widgets **empilhando-os** sequencialmente, permitindo que ocupem o espaço disponível na janela. É ideal para layouts simples, onde a ordem de empilhamento é importante.

## GRID

O gerenciador Grid utiliza uma matriz de linhas e colunas, permitindo um **posicionamento mais preciso** dos widgets. É particularmente útil para formulários e interfaces complexas, facilitando a organização visual.

```
tk.Label(root, text="Usuário:").grid(row=0, column=0)  
tk.Entry(root).grid(row=0, column=1)  
tk.Button(root, text="Entrar").grid(row=1, columnspan=2)
```

```
tk.Button(root, text="Relatórios", width=20).place(x=50, y=20,)  
tk.Button(root, text="Configurações", width=20).place(x=50, y=60)
```

## PLACE

O gerenciador **Place** permite o posicionamento absoluto dos widgets em coordenadas X e Y, oferecendo controle total sobre a localização, ideal para layouts específicos e personalizados.



# Sistema de Login

Este exemplo demonstra como criar um **sistema de login** simples utilizando o gerenciador de layout Grid no Tkinter, facilitando o posicionamento e a organização dos elementos da interface.

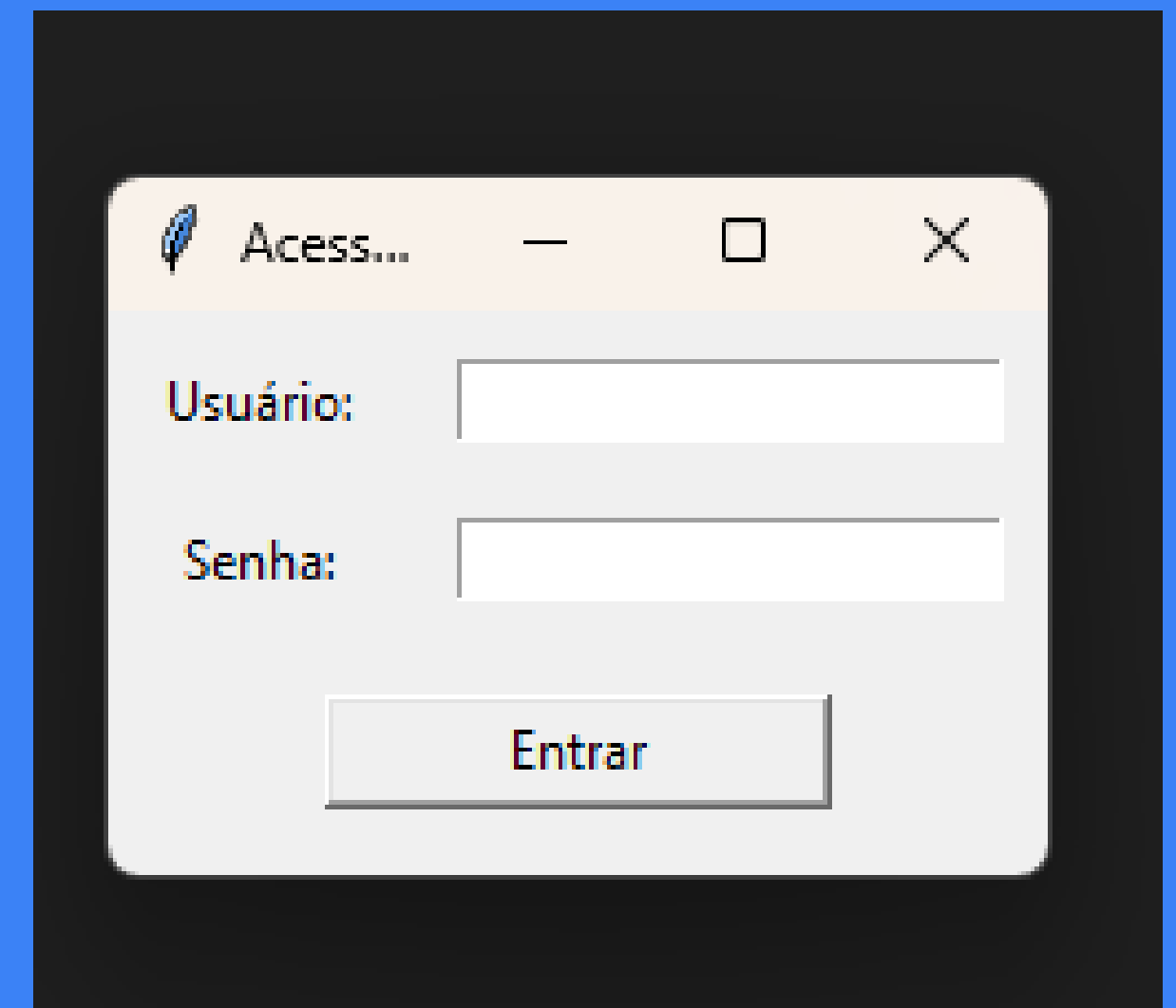
```
import tkinter as tk
root = tk.Tk()
root.title("Acesso ao Sistema")

tk.Label(root, text="Usuário:").grid(row=0, column=0, padx=10, pady=10)
tk.Entry(root).grid(row=0, column=1, padx=10, pady=10)

tk.Label(root, text="Senha:").grid(row=1, column=0, padx=10, pady=5)
tk.Entry(root, show="*").grid(row=1, column=1, padx=10, pady=5)

tk.Button(root, text="Entrar", width=15).grid(row=2, column=0, columnspan=2, pady=15)

root.mainloop()
```



# Menu Vertical

O **menu vertical** é uma forma eficaz de organizar opções em uma interface. Usando o gerenciador de layout Pack, ele permite um design intuitivo e acessível para os usuários.

```
import tkinter as tk

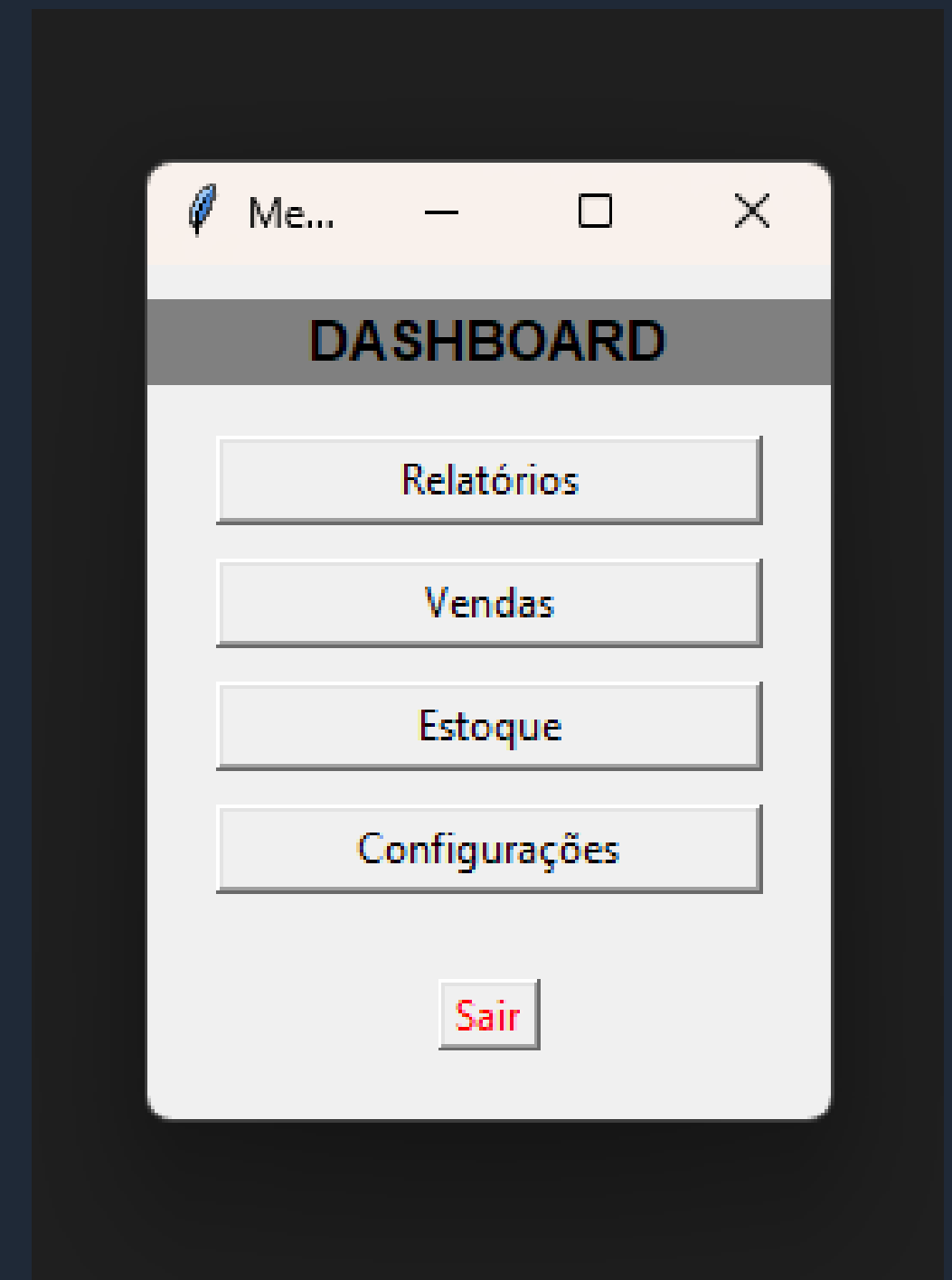
root = tk.Tk()
root.title("Menu Principal")
root.geometry("200x250")

label_dash = tk.Label(root, text="DASHBOARD", font=("Arial", 12, "bold"), bg="gray")
label_dash.pack(fill="x", pady=10)

buttons = ["Relatórios", "Vendas", "Estoque", "Configurações"]
for btn_text in buttons:
    tk.Button(root, text=btn_text).pack(fill="x", padx=20, pady=5)

tk.Button(root, text="Sair", fg="red").pack(side="bottom", pady=20)

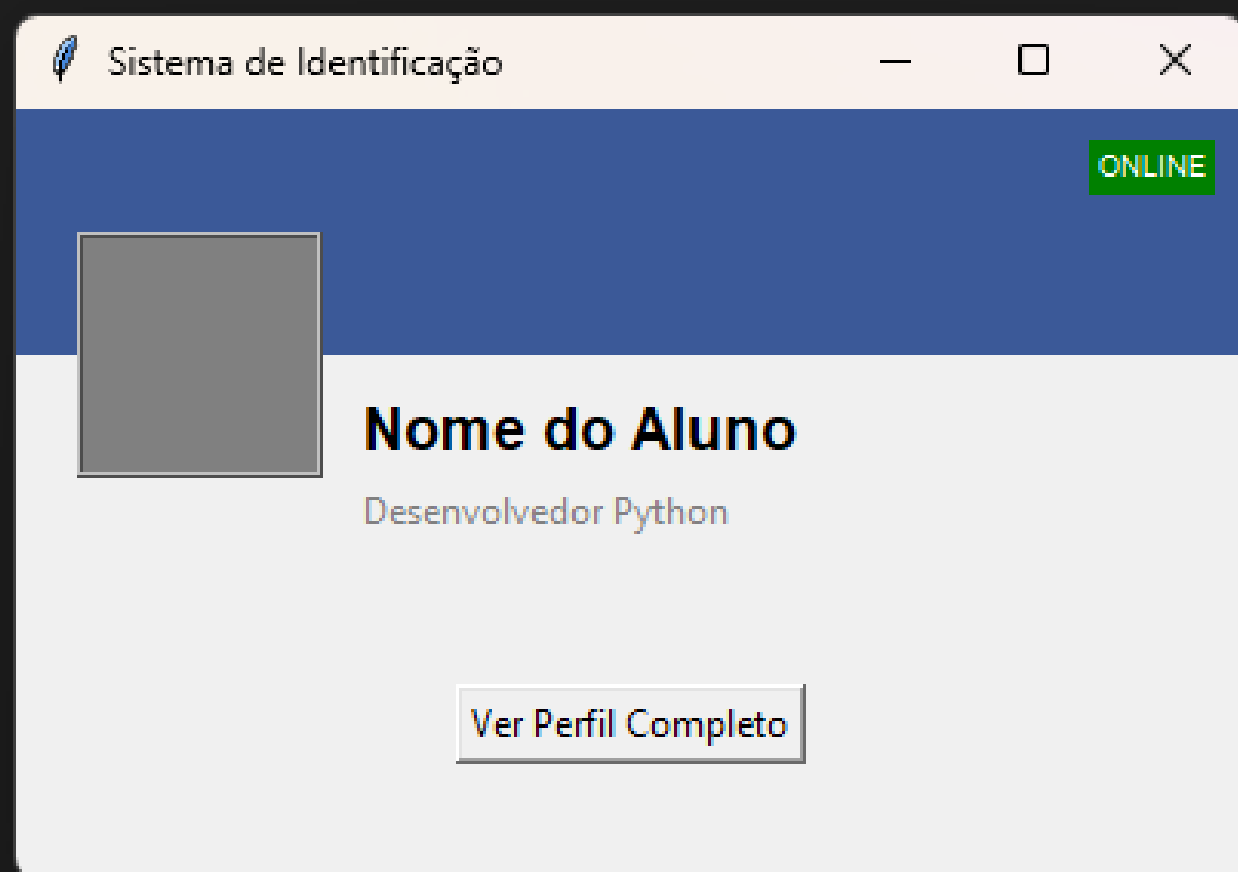
root.mainloop()
```





# Sistema de Login

O place é útil para colocar elementos em posições relativas (%) ou fixas específicas. Vamos construir um crachá digital.



```
import tkinter as tk

root = tk.Tk()
root.title("Sistema de Identificação")
root.geometry("400x250")

banner = tk.Frame(root, bg="#3b5998", height=80)
banner.place(x=0, y=0, relwidth=1.0) # Ocupa 100% da largura

foto = tk.Frame(root, bg="gray", bd=2, relief="ridge")
foto.place(x=20, y=40, width=80, height=80)

nome = tk.Label(root, text="Nome do Aluno", font=("Arial", 14, "bold"))
nome.place(x=110, y=90)

cargo = tk.Label(root, text="Desenvolvedor Python", fg="gray")
cargo.place(x=110, y=120)

btn_perfil = tk.Button(root, text="Ver Perfil Completo")
btn_perfil.place(relx=0.5, rely=0.8, anchor="center")

status = tk.Label(root, text="ONLINE", bg="green", fg="white", font=("Arial", 7))
status.place(relx=1.0, x=-10, y=10, anchor="ne")

root.mainloop()
```

# Principais Widgets

## BUTTON

O **Button** é um elemento interativo que permite ao usuário executar ações ao ser clicado. É fácil de implementar e customizar, ideal para iniciar eventos na interface.

```
botao = tk.Button(root, text="Clique Aqui", command=print("Botão acionado!"))  
botao.pack()
```

## ENTRY

O **Entry** fornece um campo de texto de linha única, permitindo ao usuário inserir dados. Ele é simples e versátil, frequentemente utilizado para capturar informações como nomes ou senhas.

```
tk.Label(root, text="Digite seu nome:").pack()  
tk.Entry(root).pack(padx=10, pady=5)
```

## TEXT

O widget **Text** cria um campo de texto multilinha, ideal para exibição ou entrada de grandes volumes de texto. Permite formatação e manipulação mais complexa, sendo útil em aplicativos variados.

```
tk.Label(root, text="Observações:").pack()  
campo_texto = tk.Text(root, height=5, width=30)  
campo_texto.pack(padx=10, pady=5)  
campo_texto.insert("1.0", "Digite sua mensagem aqui...")
```

# Principais Widgets

## CHECKBUTTON

O **Checkbutton** permite que os usuários selecionem múltiplas opções em uma interface gráfica. É ideal para configurações onde várias escolhas podem ser feitas simultaneamente.

```
tk.Checkbutton(root, text="Aceito os termos de uso").pack(pady=10)
```

## RADIOBUTTON

O **Radiobutton** é usado para selecionar uma única opção entre várias disponíveis. Ele garante que apenas uma escolha seja feita, facilitando a interação com o usuário.

```
tk.Label(root, text="Qual o seu nível de experiência?").pack()  
tk.Radiobutton(root, text="Iniciante", value=1).pack(anchor="w", padx=20)  
tk.Radiobutton(root, text="Intermediário", value=2).pack(anchor="w", padx=20)  
tk.Radiobutton(root, text="Avançado", value=3).pack(anchor="w", padx=20)
```

## LABEL

O **Label** é um widget fundamental que exibe texto ou imagens na interface. Ele é frequentemente usado para descrever outros widgets, fornecendo informações claras ao usuário.

```
tk.Label(root, text="Atenção: campos obrigatórios *", fg="red").pack()
```

# Atributos de Customização

## TEXTO

O atributo **text** define o conteúdo textual exibido em widgets como labels, buttons e outros, permitindo que você personalize a mensagem apresentada ao usuário.

## FONTE

O atributo **font** possibilita a definição da fonte e tamanho do texto, essencial para manter a legibilidade e a estética da interface gráfica de usuário.

## CORES

Os atributos **bg** e **fg** permitem a customização das cores de fundo e do texto, respectivamente, proporcionando uma experiência visual atrativa e adaptável ao tema da aplicação.

## DIMENSÕES

Os atributos **width** e **height** controlam as dimensões dos widgets, garantindo que eles sejam proporcionais e adequados ao layout, melhorando a usabilidade da interface.

# Perguntas e documentações

## 1. Documentação Oficial

- [Python.org \(Tkinter Docs\)](#): O ponto de partida oficial. Contém a explicação técnica de cada classe e método.
- [Tcl.tk \(Tk Manual\)](#): Como o Tkinter é um wrapper, o manual original do Tcl/Tk é a fonte mais completa sobre o comportamento dos widgets.

## 2. Guias Práticos e Didáticos

- [TkDocs \(tkdocs.com\)](#): Provavelmente a melhor ferramenta de aprendizado. Oferece tutoriais modernos e mostra como o código funciona em diferentes linguagens (Python, Ruby, Tcl).
- [Real Python \(realpython.com\)](#): Possui guias visuais excelentes para iniciantes, com exemplos de projetos completos.
- [GeeksforGeeks / Tutorialspoint](#): Ótimos para consultas rápidas sobre um widget específico e seus atributos.

## 3. Modernizando o Tkinter (Extensões)

- [CustomTkinter \(Pypi/GitHub\)](#): Uma biblioteca que envolve o Tkinter para criar interfaces com visual moderno, arredondado e com suporte a "Modo Escuro" nativo.
- [Pythonguides.com](#): Focado em exemplos práticos de interface e solução de problemas comuns.





# Projeto Final: Sistema de Agendamento "Pet Shop AuQmia"

## OBJETIVO DO ALUNO

Construir uma interface de formulário que capture dados variados, alinhe os campos corretamente e exiba uma confirmação.



Pet Shop AuQmia - Sistema de Agendamento

Agendamento de Serviços

Nome do Dono:

Nome do Pet:

Espécie:

☒ Cachorro

☐ Gato

Selecione os Serviços

☐ Banho Completo

☐ Tosa Higiênica

☐ Corte de Unhas

Observações Adicionais:

AGENDAR HORÁRIO

Sistema v1.0 - Dev Class